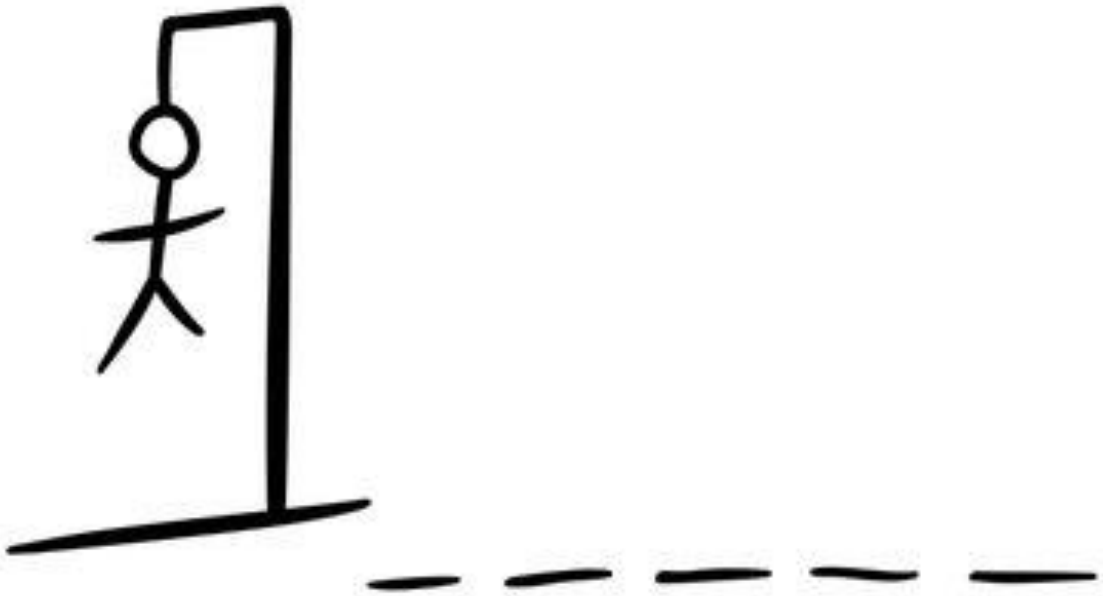


Daria Fedorova, Marcela Santos,
Simran Singh, Selvi Lesmana Putri

Mar 2, 2026

Hangman Game Project

Management Science and Operations Research

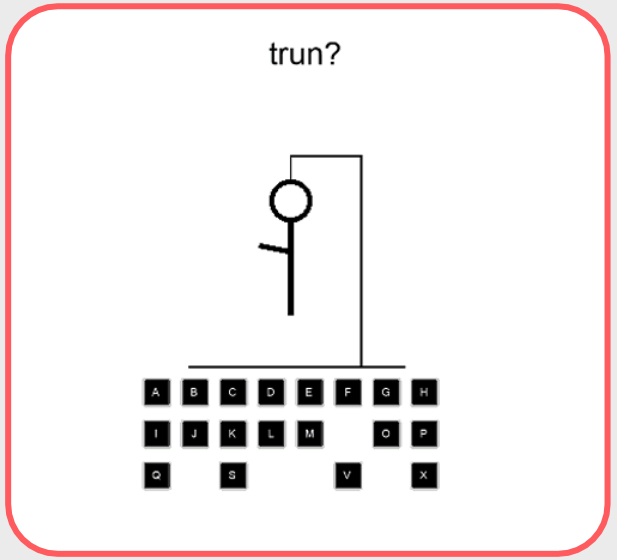
KÜHNE LOGISTICS UNIVERSITY

Leading with an Operations Mindset

Agenda

1. Game Description
2. MDP Formulation
3. Environment Setup
4. Approaches applied: Heuristics & Reinforcement Learning (Q-Learning & SARSA)
5. Comparison of Algorithms
6. Conclusion
7. Retrospective

What is Hangman Game?



Game Setup

- A hidden **5-letter word** is selected
- The word is shown as underscores: "_ _ _ __ "
- The player has a limited no. of wrong guesses (e.g., 6)

How it Works?

- Guess **one letter at a time**
- ✓ → all positions are revealed
- ✗ → remaining guesses decrease by 1
- No repeated guesses
- End when the word is solved or guesses run out

Objective

- Reveal the full word before reaching 0 wrong guesses
- Use revealed letters to narrow possibilities

Agenda

1 Game Description

2 MDP Formulation

MDP Formulation: States

{ _ _ _ _ _, (guessed letters), No of turns }

$26^5 = 11.9\ mio$

$26 \times 25 \times 24 \times 23 \times 22 \times 21 = 165.8\ mio$

$6\ turns\ in\ total\ per\ game$

MDP Formulation: Actions

{pick a letter, guess a word}

26 letters in alphabet

26^5

MDP Formulation: Outcomes

```
      +---+
      |   |
      O   |
     /|\  |
          |
          |
    =========

--------------------------
Agent WON 🎉
Final word: watch
```

```
else:
    print("Agent LOST ❌")
```

Per episode: {Agent Won, Agent Lost}

Per-step: {Correct new letter, Wrong new letter, Repeated letter}

Reward +1 *and* +5 *extra if the word is completed*

Reward -1, No of guesses is decreases by 1

State remains unchanged

MDP Formulation: Rewards

Correct Guess

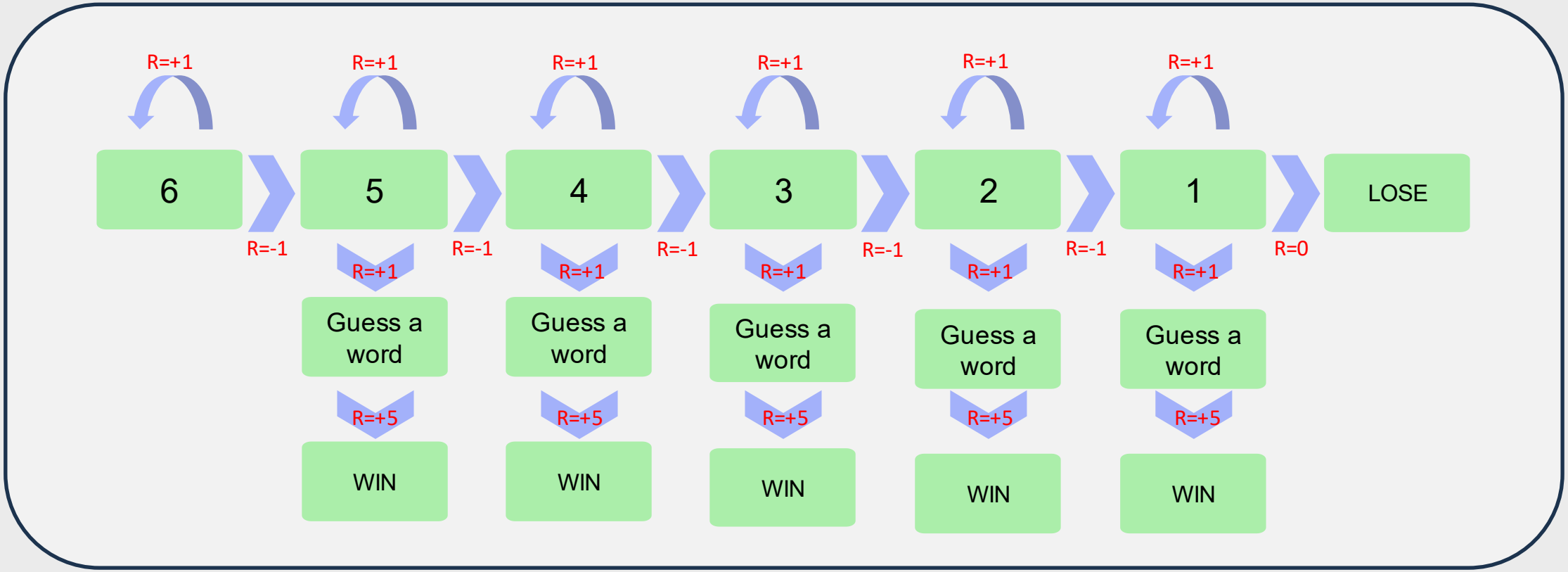
Wrong Guess

```
if action in self._word:
    reward = 1    # correct guess
else:
    reward = -1   # wrong guess
```

Win Bonus

```
if "_" not in self._pattern:
    reward += 5
```

MDP Formulation: Transition Schema



Agenda

1. Game Description
2. MDP Formulation
3. Environment Setup

Environment Setup:

WordSampler():
Determine 80/20 split ratio for training and testing of the dictionary. Seed = 42

EnvHangmanGym(): wraps the Hangman game in a Gymnasium interface, the agent sees the board as 32 numbers array.=`np.int32`

word = "plane", agent guessed "p" and "z":

obs[0:5] = [16, 0, 0, 0, 0] ← pattern ("P","_","_","_","_")

obs[5:31] = [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1] ← p=1, z=1

obs[31] = 5 ← one wrong guess used (z)

EnvHangman() :
Plain-Python environment.
Manages word, pattern, guessed letters, and wrong guesses. Returns state as a string

The Problem: Why Not Pre-Compute All States?

BFS / Pre-Compute All States

Build a graph of “EVERY Possible State” before the game starts

26^(5 letters) × 26 letters × 6 guess

= Millions of states

Memory: Gigabytes of RAM
Time: Minutes/hours to build
Scalability: Breaks with larger dictionaries

Real Time Transition

Compute the “NEXT state” only when needed via apply action() function

1 word × 1 action at a time

= O(word length) per step

Memory: Only current state in RAM
Time: Instant per step
Scalability: Works with any dictionary size

Step (Action) — The Game Loop

1. Validate Action
Is it from " a …. z" ? Not gussed yet??

2. Call "apply action()"
Compute next state in real time

3. Compute Reward
+1 / -1 / 0 Repeat

4. Check Termination
Win: No "_" left (+5 reward)
Loss: guesses left = 0

5. Return Tuple
(obs , reward, terminated , truncated , info)

Agenda

1. Game Description
2. MDP Formulation
3. Environment Setup
4. Approaches applied: Heuristics & Reinforcement Learning (Q-Learning & SARSA)

Heuristic Approach: Rule-based strategy

1. Computes overall letter frequency across that dictionary (`global_ranked_letters`) as a fallback strategy

```
self.global_ranked_letters = [ch for ch, _ in self.global_counts.most_common()]
```

2. Filters candidate words that still fit what is known

```
def _filter_candidates(self, pattern_str: str, guessed_letters: set[str]):
    revealed = set(pattern_str) - {"_"}
    wrong_letters = guessed_letters - revealed
```

3. Chooses the next letter from candidates

```
def choose_letter(self, pattern_tuple, guessed_list):
    pattern_str = self._pattern_tuple_to_str(pattern_tuple)
    guessed_letters = {str(g).lower() for g in guessed_list}
```

Reinforcement Learning: Q-Learning

Behaviour Policy: Epsilon-Greedy

The agent follows an epsilon-greedy behavior policy

With probability ε:

•Select random legal action (exploration)

Otherwise:

•Select action with highest Q-value (exploitation)

Why this is needed:

•Initially, all Q-values are zero

•Without exploration, the agent would never discover better actions

•Prevents getting stuck in suboptimal strategies

Key idea:

High ε early → exploration

Low ε later → exploitation

```python
def epsilon_greedy(Q, s_key, legal, eps):
    """Choose random legal action w.p. eps, else best legal action."""
    if random.random() < eps:
        return random.choice(legal)
    return max(legal, key=lambda a: float(Q[s_key][a]))
```

Reinforcement Learning: Q-Learning Update Rule

Target Policy: Learning Mechanism

We restrict the update to legal actions only, since in Hangman we cannot guess the same letter twice

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_a Q(s,a) - Q(s,a) \right]$$

$$\text{NewQ} - \text{value} = \text{OldQ} - \text{value} + \text{Learningrate} \times (\text{Target} - \text{Oldestimate})$$

```
next_legal = legal_action_indices(env)
target = float(r) + gamma * max_over_legal(Q, s2, next_legal)
Q[s][a] += alpha * (target - float(Q[s][a]))
```

The target consists of the immediate reward plus the discounted maximum Q-value of the next state

$$r + \gamma \max_a Q(s,a)$$

We assume that in the next state, the agent will behave optimally and select the best possible action

=

greedy estimate

We update the Q-value proportionally to the temporal difference error. Alpha controls how strongly new information influences the old estimate

Reinforcement Learning: Q-Learning Off Policy

```python
def epsilon_greedy(Q, s_key, legal, eps):
    """Choose random legal action w.p. eps, else best legal action."""
    if random.random() < eps:
        return random.choice(legal)
    return max(legal, key=lambda a: float(Q[s_key][a]))
```

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_a Q(s,a) - Q(s,a) \right]$$

```python
next_legal = legal_action_indices(env)
target = float(r) + gamma * max_over_legal(Q, s2, next_legal)
Q[s][a] += alpha * (target - float(Q[s][a]))
```

Behavior policy → ε-greedy

Target policy → greedy max

= off-policy algorithm

Learns optimal policy even while exploring

Reinforcement Learning: SARSA

Behavior Policy: Epsilon-Greedy

```python
def epsilon_greedy(Q, s_key, legal, eps):
    """Choose random legal action w.p. eps, else best legal action."""
    if random.random() < eps:
        return random.choice(legal)
    return max(legal, key=lambda a: float(Q[s_key][a]))
```

- With probability ε → random legal action
- Otherwise → action with highest Q-value
- ε decays over time

```python
# choose initial action
legal = legal_action_indices(env)
a = epsilon_greedy(Q, s, legal, eps)
```

This is the behavior policy selecting action a

Key difference from Q-learning:
SARSA learns using the same ε-greedy policy it behaves with

Reinforcement Learning: SARSA Update Rule

$$Q(s,a) \leftarrow Q(s,a) + \alpha\,[r + \gamma Q(s,a) - Q(s,a)]$$

$$\text{New Q} - \text{value} = \text{Old Q} - \text{value} + \text{Learning rate} \times (\text{Target} - \text{Old estimate})$$

$$\text{target} = r + \gamma Q(s,a)$$

- Immediate reward
- Plus discounted value of next chosen action
- No max operator

```
next_legal = legal_action_indices(env)
a2 = epsilon_greedy(Q, s2, next_legal, eps)

target = float(r) + gamma * float(Q[s2][a2])
Q[s][a] += alpha * (target - float(Q[s][a]))

s, a = s2, a2
```

- Selects the next action
- Uses the same epsilon-greedy policy
- Based on the next state s2

This action is not assumed to be optimal — it is the action actually taken

Unlike Q-learning:
It does NOT use max(Q[s2])
It uses the Q-value of the action actually selected

- Move forward in the episode
- The next state becomes the current state
- The next action becomes the current action

TD Update
- Adjust Q-value toward the target
- Step size controlled by learning rate \alpha

Reinforcement Learning: SARSA On Policy

```
next_legal = legal_action_indices(env)
a2 = epsilon_greedy(Q, s2, next_legal, eps)

target = float(r) + gamma * float(Q[s2][a2])
Q[s][a] += alpha * (target - float(Q[s][a]))

s, a = s2, a2
```

- Select next action a' using ε-greedy
- Compute target:

$$r + \gamma Q(s, a)$$

- Update Q-value toward this target

Behavior policy = Target policy

Learning is based on the action actually taken

= on-policy algorithm

Therefore, learning reflects the risk introduced by exploration

Hyperparameter Tuning: Experimental Design

- Training episodes: 50,000
- Evaluation episodes: 1,000 (test split)
- Separate train/test word sets (Evaluation performed on unseen words)

- Discount factor: $\gamma = 0.95$
- ε starts at 1.0 and decays linearly to 0.05 over 60% of training

- Learning rate α tuned via grid search
- Each configuration repeated with 5 random seeds
- Reported results: mean ± standard deviation

```python
def train_qlearning(
    episodes=50000,
    alpha=0.1,
    gamma=0.95,
    seed=42,
    split="train",
```

```python
def train_sarsa(
    episodes=50000,
    alpha=0.1,
    gamma=0.95,
    seed=42,
    split="train",
```

```python
def grid_search_alpha(
    alphas=(0.01, 0.05, 0.1, 0.2, 0.3),
    train_episodes=50000,
    eval_episodes=1000,
    seeds=(0, 1, 2, 3, 4),
    split_train="train",
    split_eval="test",
```

Hyperparameter Tuning: Alpha Grid Results

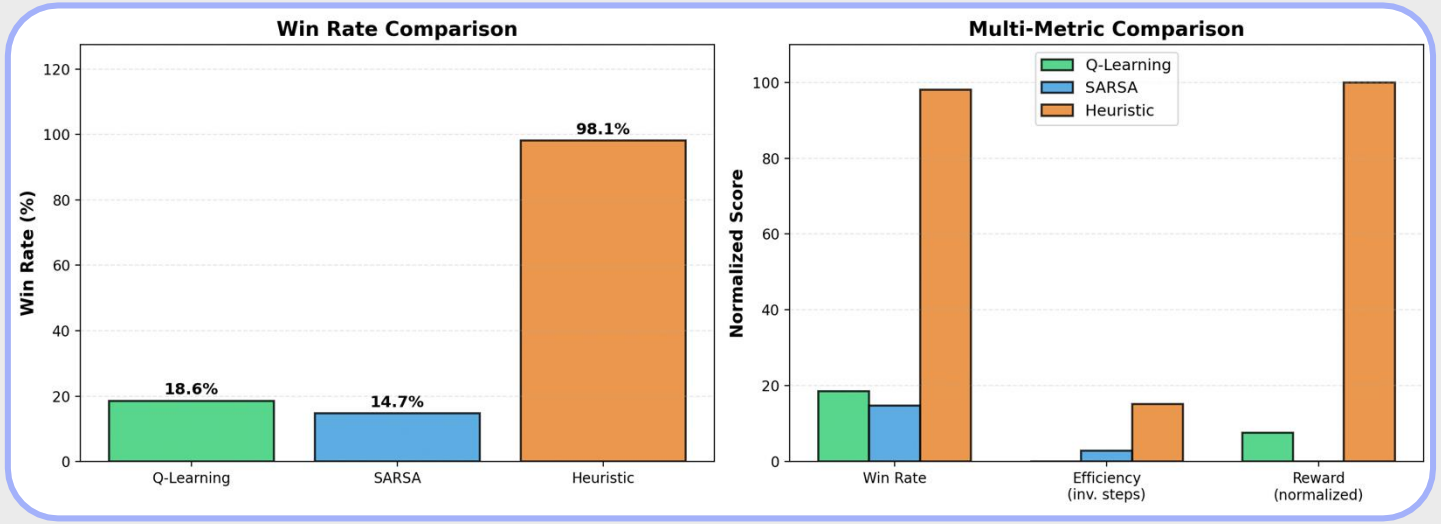
```
=== Alpha grid search (win rate on test) ===
Train episodes: 50000 | Eval episodes: 1000 | Seeds: [0, 1, 2, 3, 4]
alpha | Q-learning mean±std | SARSA mean±std
------|--------------------|---------------
0.01 |  0.343±0.036  |  0.345±0.044
0.05 |  0.360±0.085  |  0.303±0.034
0.10 |  0.247±0.069  |  0.161±0.079
0.20 |  0.096±0.021  |  0.093±0.029
0.30 |  0.041±0.013  |  0.031±0.012

Best Q-learning alpha: 0.05 (mean win rate 0.360)
Best SARSA alpha: 0.01 (mean win rate 0.345)
```

- Choose hyperparameter to tune: learning rate (α)
- Tested 5 values: 0.01, 0.05, 0.10, 0.20, 0.30
- Ran each 5 times with different random seeds
- Measured: win rate on 1,000 test episodes
- Winner: α = 0.05 for Q-Learning, α = 0.01 for SARSA

Agenda

1. Game Description
2. MDP Formulation
3. Environment Setup
4. Approaches applied: Heuristics & Reinforcement Learning (Q-Learning & SARSA)
5. Comparison of Algorithms

Model Comparison




```
Training configuration:
  Episodes: 50,000
  Evaluation games: 1,000 (test set)
Performance Comparison Table
Metric                    Q-Learning        SARSA    Heuristic
------------------------------------------------------------
Win Rate (%)                   18.60        14.70        98.10
Avg Steps per Game              8.60         8.34         7.29
Avg Reward per Game            -1.14        -1.81         7.01
States Explored               218279       177341          N/A
```

- Heuristic dominates both RL methods : **98.1% win rate** vs 18.6% (Q-Learning) and 14.7% (SARSA)
- Heuristic produces **strong positive reward (+7.01)** while both RL methods remain negative (Q: −1.14, SARSA: −1.81)
- Heuristic achieves the best efficiency: **7.29 steps/game,** compared to 8.60 (Q-Learning) and 8.34 (SARSA)
- Q-Learning explores **23%** more states

Agenda

1. Game Description
2. MDP Formulation
3. Environment Setup
4. Approaches applied: Heuristics & Reinforcement Learning (Q-Learning & SARSA)
5. Comparison of Algorithms
6. Conclusion

Conclusion

Our best guess is: Q-Learning

Measures	Q-Learning (Off-policy)	Sarsa (On-Policy)	Heuristic
Policy learned	Optimal	Actual	Predefined rules (not learned)
Action used	Best action	Actual action	Greedy heuristic action
Exploration rate	Aggressive exploration: **218,279** states explored	Conservative learning: **177,341** states explored	N/A
Reward rate	**Medium reward: -1.14**	**Lowest rewards: -1.81**	**Highest Reward: 7.01**
Win rate	**18.6%**	**14.7%**	**98.1%**

Agenda

1. Game Description
2. MDP Formulation
3. Environment Setup
4. Approaches applied: Heuristics & Reinforcement Learning (Q-Learning & SARSA)
5. Comparison of Algorithms
6. Conclusion
7. Retrospective

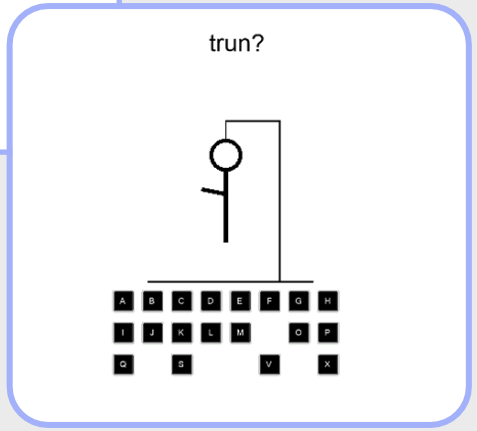
Retrospective

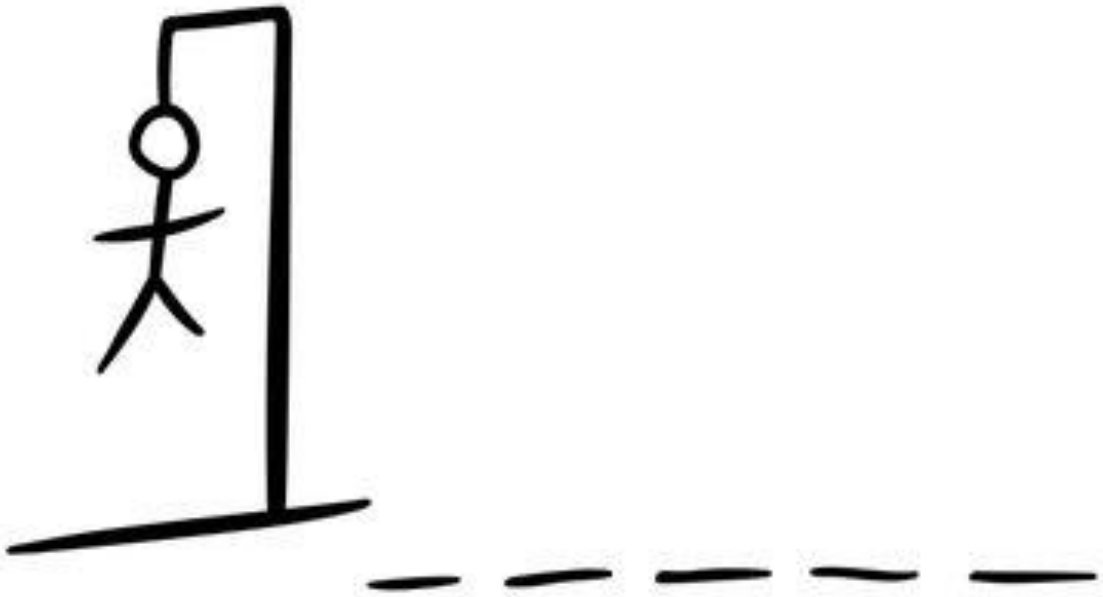
Challenges:

- Hyperparameter underexplored (epsilon ε, discount factor γ, training duration)
- Limited to 5-letter words and 1,000 words dictionary

Next Steps:

- Try Backward Pass
- Test different exploration strategies to find optimal balance between random guessing and using learned knowledge to improve the performance
- Expand to use 6-7 letter words and 2K – 5K words for dictionary





Any questions left?

KÜHNE LOGISTICS UNIVERSITY

Leading with an Operations Mindset